

Absolutely. The next step is to turn the architecture from v1.1 into a concrete backend engineering specification. I'll keep this implementation-oriented, with the Icelandic legal/document-management requirements treated as system constraints rather than hard-coding assumptions about legal practice.

ILCMS — Icelandic Legal Case Management System

Technical Specification v1.2 — Backend Implementation Specification

Document version: 1.2

Status: Engineering Specification

Language: English

Target market: Iceland

Primary backend stack: Python / FastAPI / PostgreSQL

Related specification: Technical Specification v1.1 — DevOps, CI/CD & Kubernetes Implementation Specification

1. Purpose

This specification defines the implementation architecture and engineering standards for the ILCMS backend.

The backend shall provide the authoritative application services for:

- user and organization management
- authentication and authorization
- case management
- parties and contacts
- legal matters and classifications
- documents and document metadata
- document versions
- case workflow
- tasks and deadlines
- notes and communications

- search
- audit logging
- notifications
- background processing
- integrations
- AI-assisted functions where enabled

The backend shall expose a versioned REST API consumed by the ILCMS frontend and potentially by external integrations.

The backend shall be designed so that sensitive legal information remains protected throughout its entire lifecycle.

2. Backend Design Principles

The implementation shall follow these principles:

1. **Security by default**
2. **Explicit authorization**
3. **Strong tenant isolation**
4. **Domain-driven organization of business logic**
5. **API-first development**
6. **Database integrity**
7. **Immutable auditability**
8. **Asynchronous processing for expensive operations**
9. **Idempotent background jobs**
10. **Explicit transaction boundaries**
11. **No business logic in API controllers**
12. **No direct database access from API controllers**
13. **No secrets in source code**
14. **No sensitive data in ordinary application logs**
15. **UTC internally, Icelandic/local timezone handling at the presentation boundary**
16. **Icelandic characters and names treated as first-class data**

- 17. Backward-compatible API evolution
- 18. Testability of every business-critical operation

3. Technology Stack

3.1 Required technologies

Component	Technology
Language	Python 3.12+
API framework	FastAPI
ASGI server	Uvicorn
ORM	SQLAlchemy 2.x
Validation	Pydantic v2
Database	PostgreSQL
Migrations	Alembic
Authentication	OAuth 2.0 / OpenID Connect compatible
Cache	Redis
Message broker	RabbitMQ
Object storage	S3-compatible storage
HTTP client	httpx
Testing	pytest
API testing	httpx / pytest
Static analysis	Ruff
Type checking	mypy
Password hashing where required	Argon2id
Structured logging	JSON
Metrics	Prometheus-compatible
Tracing	OpenTelemetry

The precise dependency versions shall be pinned through the project's dependency-management mechanism and updated through controlled dependency-management processes.

4. Python Runtime

Python 3.12 shall be the initial baseline unless the deployment platform requires a newer supported version.

The project shall:

- use type hints throughout the codebase
- use `pyproject.toml`
- enforce formatting and linting
- avoid deprecated Python APIs
- avoid global mutable state
- use asynchronous programming where appropriate
- use dependency injection for external services
- maintain deterministic dependency versions

Example baseline:

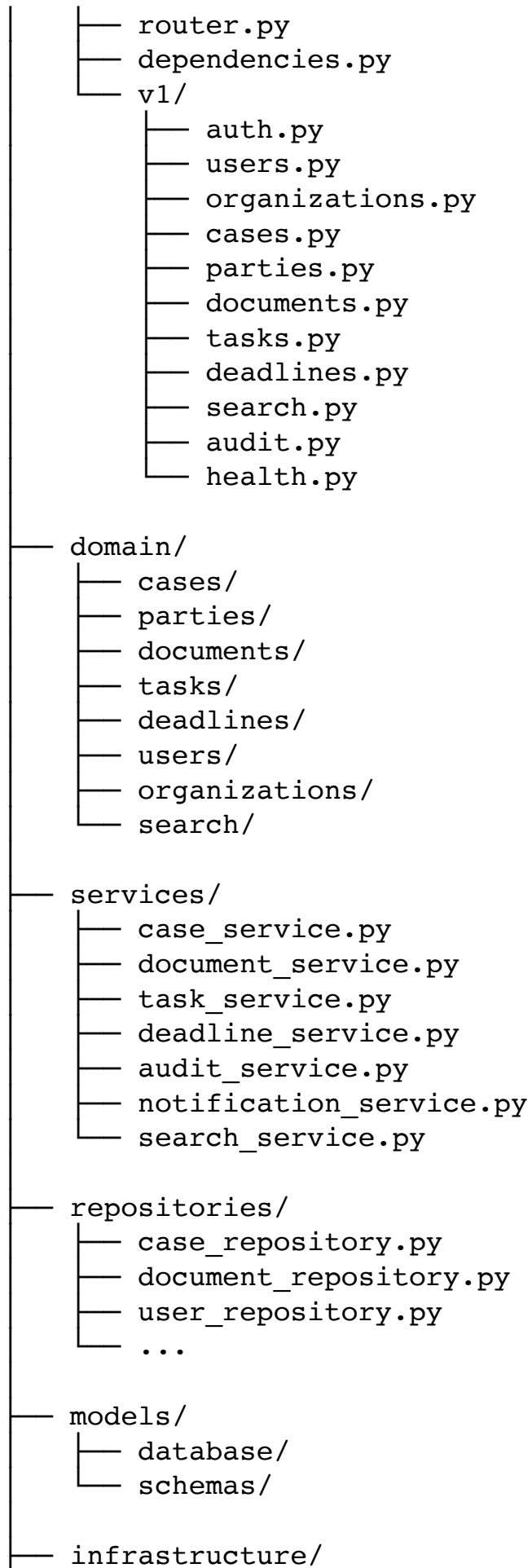
```
Python 3.12
FastAPI
Pydantic 2.x
SQLAlchemy 2.x
Alembic
asyncpg
httpx
pytest
```

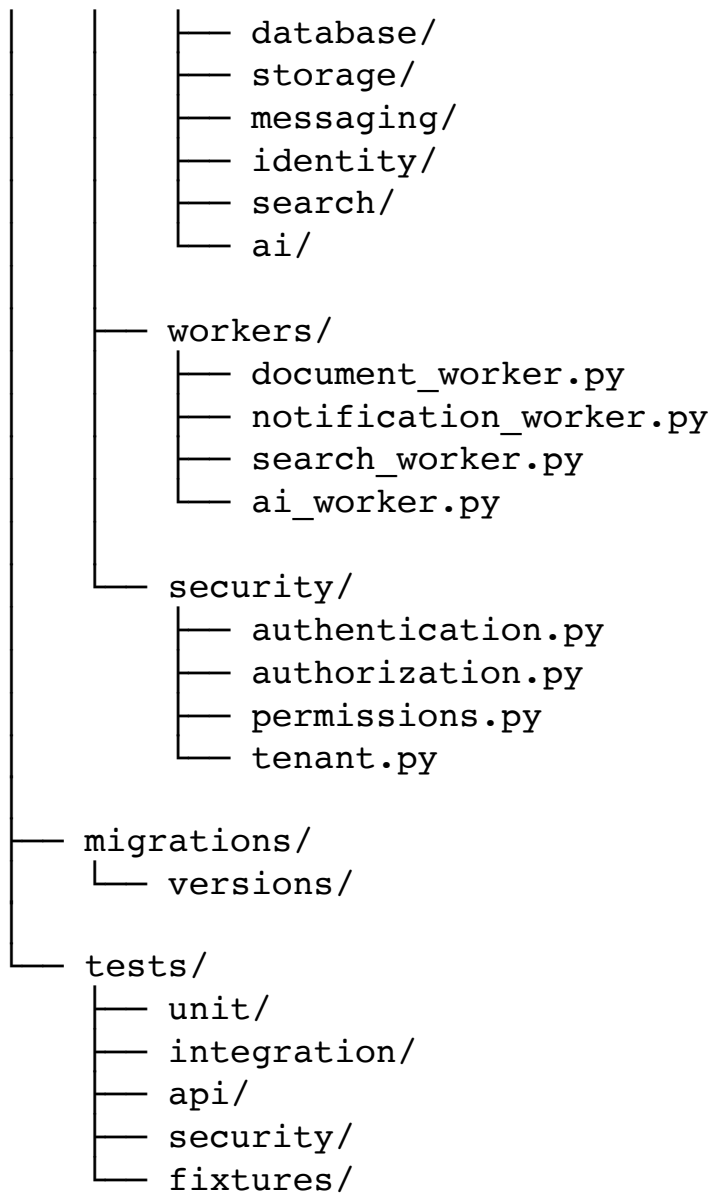
5. Repository Structure

The backend shall use a modular structure.

Recommended structure:

```
backend/
├── pyproject.toml
├── README.md
├── alembic.ini
├── app/
│   ├── main.py
│   ├── config.py
│   ├── logging.py
│   └── api/
```

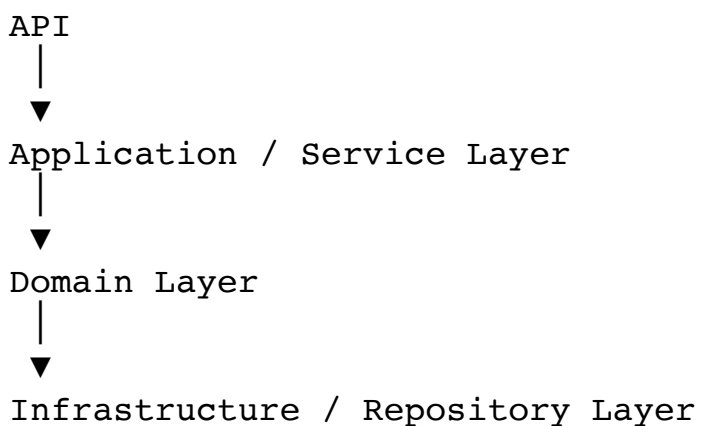




The precise directory structure may evolve, but the architectural separation shall remain.

6. Architectural Layers

The backend shall use four principal layers.





External Systems / Database

6.1 API layer

Responsible for:

- HTTP routing
- authentication dependencies
- request validation
- response serialization
- HTTP status codes
- API documentation

The API layer shall not contain substantive business rules.

6.2 Application/service layer

Responsible for:

- orchestration
- transaction boundaries
- business use cases
- authorization checks
- invoking repositories
- generating audit events
- publishing domain events

Example:

```
create_case()  
close_case()  
assign_case()  
upload_document()  
approve_document()  
create_task()  
complete_task()
```

6.3 Domain layer

Contains business concepts and rules.

Examples:

Case

Party

Document

DocumentVersion

Task

Deadline

User

Organization

Permission

CaseStatus

DocumentStatus

Domain logic shall remain independent of FastAPI and database implementation details where practical.

6.4 Infrastructure layer

Responsible for:

- PostgreSQL
- Redis
- RabbitMQ
- object storage
- identity providers
- search infrastructure
- external APIs
- AI services

Infrastructure implementations shall be replaceable through interfaces where practical.

7. Configuration Management

Configuration shall be provided through environment variables and/or the deployment configuration layer.

Example:

DATABASE_URL

REDIS_URL
RABBITMQ_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
OIDC_ISSUER
OIDC_CLIENT_ID
OIDC_CLIENT_SECRET
VAULT_ADDR
LOG_LEVEL
ENVIRONMENT

Secrets shall never be stored in:

- Git
- Dockerfiles
- Kubernetes manifests
- application source code
- frontend source code
- CI configuration in plaintext

Secrets shall be supplied through the secret-management mechanism defined in v1.1.

8. Database Architecture

PostgreSQL shall be the authoritative transactional datastore.

The database shall use:

- UUID identifiers
- foreign-key constraints
- explicit indexes
- unique constraints
- check constraints where appropriate
- transactions
- timestamps
- soft deletion where legally/business appropriate
- database-level tenant isolation where appropriate

Example:

```
id UUID PRIMARY KEY
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

9. Identifier Strategy

Externally exposed entities shall use UUIDs rather than sequential database IDs.

Example:

```
case_id:
550e8400-e29b-41d4-a716-446655440000
```

This reduces enumeration risk and simplifies distributed-system design.

Human-readable case numbers shall be separate from internal identifiers.

Example:

```
Internal ID:
UUID
```

```
Human case number:
2026-00157
```

10. Multi-Tenancy

The system shall support organizations/tenants.

Every tenant-owned resource shall have an explicit organization/tenant relationship.

Example:

```
Organization
├── Users
├── Cases
├── Documents
├── Tasks
└── Audit Events
```

Tenant identity shall never be accepted solely from a client-supplied request parameter.

It shall be derived from authenticated identity and authorized tenant membership.

11. Authorization

Authorization shall be evaluated server-side for every protected operation.

The system shall support RBAC initially.

Example roles:

SYSTEM_ADMIN
ORGANIZATION_ADMIN
LAWYER
PARALEGAL
CASE_MANAGER
ASSISTANT
READ_ONLY
AUDITOR

Permissions shall be more granular than roles.

Example:

case.read
case.create
case.update
case.delete
case.assign

document.read
document.upload
document.update
document.download
document.delete

task.create
task.assign
task.complete

audit.read

Roles shall map to permissions.

12. Object-Level Authorization

RBAC alone is insufficient.

The backend shall verify that the authenticated user has access to the specific object.

Example:

User
└─ Organization A
 └─ Case 123

A user belonging to Organization A must not automatically gain access to every sensitive case if additional restrictions apply.

The authorization service shall therefore support:

User

+ Organization

+ Role

+ Permission

+ Resource

+ Resource ownership/access rules

This provides a foundation for future ABAC.

13. Authentication

Authentication shall preferably be delegated to an OIDC-compatible identity provider.

The backend shall validate:

- issuer
- audience
- signature
- expiration
- required claims
- tenant membership
- authentication assurance where required

The backend shall not implement its own password system unless there is a specific business requirement.

Where local passwords are required, passwords shall use Argon2id.

14. API Versioning

All public APIs shall be versioned.

Initial version:

/api/v1/

Examples:

GET /api/v1/cases

POST /api/v1/cases
GET /api/v1/cases/{case_id}
PATCH /api/v1/cases/{case_id}
Breaking changes shall require a new API version.

15. HTTP Standards

The API shall use conventional HTTP semantics.

Examples:

Operation	Method
List	GET
Retrieve	GET
Create	POST
Partial update	PATCH
Replace	PUT where appropriate
Delete	DELETE

Typical responses:

200 OK
201 Created
202 Accepted
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
429 Too Many Requests
500 Internal Server Error

16. Standard Error Format

All API errors shall use a consistent structure.

Example:

```
{  
  "error": {
```

```
    "code": "CASE_NOT_FOUND",
    "message": "The requested case could not be found.",
    "request_id": "...",
  }
}
```

Sensitive internal information shall never be returned.

Database exceptions, stack traces and infrastructure details shall not be exposed to clients.

17. Request Correlation

Every request shall have a correlation/request ID.

Example:

X-Request-ID

If supplied by a trusted upstream system, it may be propagated.

Otherwise the backend shall generate one.

The request ID shall appear in:

- application logs
- audit-related technical metadata
- tracing information
- error responses where appropriate

18. Core Case Domain

The Case entity is the central business object.

Minimum conceptual attributes:

```
id
case_number
organization_id
title
description
case_type
classification_id
status
priority
opened_at
closed_at
```

created_by
responsible_user_id
created_at
updated_at
Potential statuses:

DRAFT
OPEN
ACTIVE
ON_HOLD
CLOSED
ARCHIVED

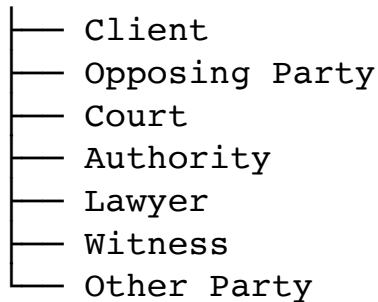
The exact legal/business meaning of each status shall be configurable.

19. Parties

A case may contain multiple parties.

Conceptual model:

Case



The system shall not assume that a party is necessarily a natural person.

Party types should include:

PERSON
ORGANIZATION
AUTHORITY
COURT
OTHER

Names shall support Icelandic characters, including:

Á É Í Ó Ú Ý Þ Æ Ö Ð
and lowercase equivalents.

20. Icelandic Personal Names

The system shall not assume conventional Western first-name/last-name semantics.

The data model should therefore support:

display_name
given_name
family_name
full_legal_name
without requiring family_name to exist.

Search and sorting shall be designed to support Icelandic naming conventions.

21. Case Classification

The backend shall support a configurable classification system.

Conceptually:

Classification

- code
- name
- description
- parent_id
- active

This provides the foundation for an Icelandic málalykill-compatible classification structure.

The classification system shall not be hard-coded to a single customer's taxonomy.

22. Case Registration

The backend shall support the minimum metadata necessary for systematic case registration.

Conceptual fields include:

case opening date
case ID
classification
parties
case title
responsible person
status

Document metadata shall similarly support:

received date
document date
classification
sender

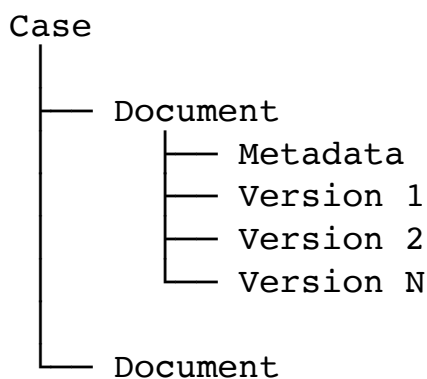
recipient
subject
case ID

The exact mandatory fields may vary according to organization and applicable records-management requirements.

23. Documents

Documents shall be treated as first-class entities.

Conceptual structure:



A document record shall contain metadata independent of the physical object-storage file.

24. Document Entity

Example conceptual model:

Document

id
case_id
organization_id
title
document_type
classification_id
status
sender
recipient
document_date
received_at
created_by
created_at
updated_at

Binary files shall not normally be stored directly inside PostgreSQL.

They shall be stored in object storage.

25. Document Versions

Each modification to a document shall create a new version where versioning is required.

Example:

Document

Version 1

Version 2

Version 3

Versions should contain:

id

document_id

version_number

object_storage_key

checksum

mime_type

size

created_by

created_at

Existing versions shall be immutable.

26. Document Integrity

Each stored document version shall have a cryptographic checksum.

Recommended algorithm:

SHA-256

Example:

```
checksum = SHA256(file_bytes)
```

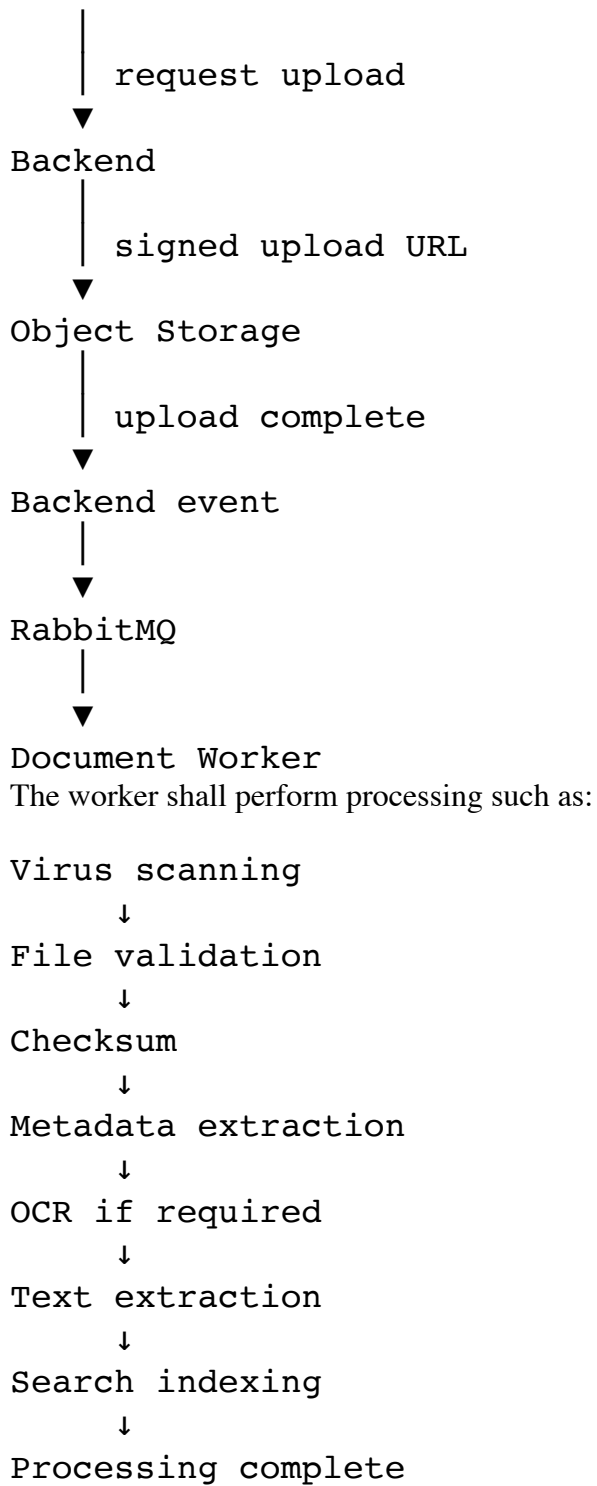
The checksum shall be usable to verify that a document has not been altered.

27. Document Upload Workflow

The API shall not require large documents to pass through the application server unnecessarily.

Preferred flow:

Frontend



28. Document Security

The backend shall verify:

- MIME type
- extension
- file size

- checksum
- storage authorization
- malware scan result

Client-supplied MIME type shall not be trusted.

Where practical, file content shall be inspected to determine the actual file type.

29. Document Download

Document downloads shall require authorization.

The preferred mechanism is a short-lived signed URL.

Example workflow:

GET /api/v1/documents/{id}/download



Authorization



Generate short-lived URL



Client downloads from object storage
The URL shall have a short expiration time.

30. Audit Logging

Audit events are a fundamental system requirement.

Security-sensitive and legally significant operations shall generate audit events.

Examples:

CASE_CREATED

CASE_UPDATED

CASE_CLOSED

CASE_ASSIGNED

DOCUMENT_UPLOADED

DOCUMENT_DOWNLOADED

DOCUMENT_VERSION_CREATED

DOCUMENT_DELETED

USER_CREATED

USER_ROLE_CHANGED

PERMISSION_CHANGED

LOGIN_SUCCESS

LOGIN_FAILURE

DATA_EXPORTED

31. Audit Event Structure

Conceptual model:

AuditEvent

id

organization_id

actor_user_id

event_type

resource_type

resource_id

timestamp

request_id

ip_address

user_agent

metadata

Audit events shall be append-only.

Normal users shall not be able to modify or delete audit records.

32. Audit Integrity

The implementation should support tamper-evident audit storage.

Possible future mechanism:

event N

↓

hash(N)

↓

hash(N + previous_hash)

↓

...

Alternatively, audit events may be exported to an immutable/WORM-compatible storage system.

The exact implementation shall be finalized during the security implementation phase.

33. Tasks

Tasks shall be first-class entities.

Example:

```
Task
----
id
case_id
title
description
assigned_to
created_by
priority
status
due_at
completed_at
created_at
updated_at
Statuses:
```

```
OPEN
IN_PROGRESS
BLOCKED
COMPLETED
CANCELLED
```

34. Deadlines

Deadlines shall be represented separately from ordinary tasks where legal/business significance requires it.

Example:

```
Deadline
-----
id
case_id
title
description
```

due_at
status
source
created_by
completed_at

The system shall distinguish between:

Task deadline
Legal/business deadline
Internal reminder

This distinction is important because not every system reminder represents a legally binding deadline.

35. Time Handling

All backend timestamps shall use timezone-aware timestamps.

PostgreSQL:

`TIMESTAMPTZ`
Application representation:

`datetime with timezone`
UTC shall be used internally wherever practical.

Presentation shall support Icelandic local time.

The system shall not assume that Iceland uses daylight-saving-time changes.

36. Notifications

Notifications shall be asynchronous.

Examples:

Task assigned
Deadline approaching
Document processing completed
Case assigned
Document upload failed
Workflow:

Business event

↓

RabbitMQ

↓

Notification worker

↓

Email / in-app notification

Notification delivery shall be retryable and idempotent.

37. Background Jobs

Long-running operations shall not block API requests.

Examples:

OCR

Virus scanning

Text extraction

Search indexing

Email delivery

Report generation

AI processing

Bulk exports

API example:

POST /api/v1/documents/{id}/process

Response:

202 Accepted

The client can then query processing status.

38. Job State

Background jobs should maintain explicit state.

Example:

QUEUED

RUNNING

COMPLETED

FAILED

RETRYING

CANCELLED

A job should have:

job_id

job_type

resource_id

status

attempt_count
created_at
started_at
completed_at
error_code

39. Idempotency

Operations that may be retried shall be idempotent.

This is particularly important for:

- document processing
- notifications
- external integrations
- payments if introduced later
- imports
- exports
- event consumers

The system shall use idempotency keys where appropriate.

Example:

Idempotency-Key: <uuid>

40. Messaging

RabbitMQ shall be used for asynchronous application events and jobs where defined by v1.1.

Messages shall contain:

```
{  
  "event_id": "...",  
  "event_type": "DOCUMENT_UPLOADED",  
  "occurred_at": "...",  
  "organization_id": "...",  
  "resource_id": "...",  
  "schema_version": 1  
}
```

Consumers shall validate message schemas.

41. Message Processing

Consumers shall implement:

Receive

↓

Validate

↓

Authorize/context-check

↓

Process

↓

Commit

↓

Acknowledge

Failures shall use retry policies and dead-letter queues.

Messages shall not contain unnecessary confidential document content.

Prefer identifiers and references over large sensitive payloads.

42. Search

Search shall support at least:

- case number
- case title
- party name
- document title
- document metadata
- extracted document text
- classification
- status
- dates
- assigned user

The implementation shall support Icelandic text.

The search architecture should allow future integration with a dedicated search engine where PostgreSQL full-text search becomes insufficient.

43. Icelandic Search

Search must correctly handle:

- UTF-8
- Icelandic characters
- case-insensitive matching
- appropriate normalization
- stemming/tokenization where supported
- names
- legal terminology

Search must avoid corrupting or incorrectly normalizing Icelandic names.

Search indexes shall be treated as derived data and must be rebuildable from authoritative database/object-storage content.

44. API Pagination

Collection APIs shall be paginated.

Example:

```
GET /api/v1/cases?page=1&page_size=50
```

A cursor-based implementation may be preferred for very large datasets.

Responses shall contain pagination metadata.

Example:

```
{
  "items": [],
  "page": 1,
  "page_size": 50,
  "total": 1234
}
```

45. Filtering and Sorting

Collection APIs shall support controlled filtering.

Example:

```
GET /api/v1/cases?  
    status=OPEN&  
    assigned_to=<uuid>&  
    classification=<code>
```

Sorting fields shall be explicitly whitelisted.

Clients shall not be permitted to inject arbitrary SQL expressions through sort/filter parameters.

46. Database Transactions

Business operations requiring multiple database changes shall execute within an explicit transaction.

Example:

```
Create case  
↓  
Create parties  
↓  
Create audit event  
↓  
Publish event/outbox  
↓  
Commit
```

The transaction shall either complete successfully or roll back.

47. Transactional Outbox

Where reliable event publication is required, the backend should implement the transactional outbox pattern.

Example:

```
Database transaction  
├─ business data  
└─ outbox event  
    ↓  
  commit  
    ↓  
Outbox publisher  
    ↓  
  RabbitMQ
```

This prevents the failure scenario where database changes succeed but event publication fails.

48. API Security

The API shall implement:

- authentication
- authorization
- input validation
- rate limiting where appropriate
- request-size limits
- secure headers
- CORS restrictions
- CSRF protection where cookie-based authentication is used
- protection against injection attacks
- safe error handling

49. Sensitive Data

Sensitive information shall be classified.

Examples include:

Personal information

Legal case information

Confidential communications

Authentication information

Documents

Security credentials

The application shall avoid:

logging document contents

logging passwords

logging tokens

logging session cookies

logging secret values

logging unnecessary personal information

50. Data Export

Exports shall be authorization-controlled.

Examples:

Case export

Document package

Audit export

Reporting export

Large exports shall execute asynchronously.

Every sensitive export shall generate an audit event.

51. Deletion

Deletion shall not be implemented as a simple unrestricted database DELETE for legally significant records.

The application shall distinguish between:

logical deletion

archiving

retention

destruction

Retention/destruction rules shall be configurable and shall be implemented according to the organization's applicable legal and records-management requirements.

52. API Documentation

FastAPI shall generate OpenAPI documentation.

The API specification shall include:

- endpoints
- request schemas
- response schemas
- authentication requirements
- error responses
- examples
- enumerations

The OpenAPI specification shall be version-controlled or reproducibly generated during CI.

53. Health Endpoints

The backend shall expose separate endpoints for:

`/live`

`/ready`

Liveness should answer:

Is the process running?

Readiness should answer:

Can this instance safely receive traffic?

Readiness may verify required dependencies such as PostgreSQL.

It should not unnecessarily fail because an optional service is unavailable.

54. Observability

The backend shall emit:

Logs

Structured JSON logs.

Metrics

Examples:

`http_requests_total`

`http_request_duration_seconds`

`database_connection_errors`

`background_jobs_total`

`background_job_failures_total`

`document_processing_duration_seconds`

Traces

OpenTelemetry traces should cover:

`HTTP request`

↓

`service`

↓

`database`

↓

RabbitMQ
↓
external API

55. Performance Targets

Initial targets should include:

Simple authenticated API request:
< 500 ms target

Database-backed normal request:
< 1 s target

Search request:
< 2 s target

Large document processing:
asynchronous
These are engineering targets, not contractual SLAs.

Performance testing shall establish realistic production thresholds.

56. Testing Strategy

Testing shall exist at several levels.

Unit tests
Integration tests
API tests
Security tests
Database tests
Worker tests
End-to-end tests
Performance tests

57. Unit Tests

Unit tests shall cover:

- domain rules
- service logic
- permission checks

- validation
- state transitions
- deadline calculations
- document state handling

Unit tests should not require a live PostgreSQL instance unless specifically testing integration behavior.

58. Integration Tests

Integration tests shall verify:

- PostgreSQL
- migrations
- repositories
- object storage
- RabbitMQ
- Redis
- external identity integration where practical

CI shall use disposable infrastructure such as containers for integration tests.

59. Security Tests

Security tests shall verify:

Unauthenticated access denied

Unauthorized role denied

Cross-tenant access denied

Object-level access denied

Expired token denied

Invalid token denied

Special emphasis shall be placed on IDOR/BOLA vulnerabilities.

Example:

User A requests Case B

↓

403/404

The system must never accidentally return Case B.

60. State-Machine Testing

Important state transitions shall be explicitly tested.

Example:

DRAFT

↓

OPEN

↓

ACTIVE

↓

CLOSED

↓

ARCHIVED

Invalid transitions must be rejected.

For example:

ARCHIVED → ACTIVE

shall not occur unless explicitly supported by a defined business rule.

61. Database Migration Strategy

Alembic shall manage schema migrations.

Production migrations shall:

- be reviewed
- be tested
- be backward-compatible where possible
- avoid unnecessary table locks
- support rollback strategy where practical

Destructive migrations shall use an expand/contract strategy.

Example:

Release 1:

Add new column

Release 2:
Populate/use new column

Release 3:
Remove old column

62. Dependency Injection

FastAPI dependency injection shall be used for:

- database sessions
- authenticated user
- authorization context
- repositories
- services
- configuration
- external clients

Example conceptual dependency chain:

```
Request
  ↓
CurrentUser
  ↓
AuthorizationContext
  ↓
CaseService
  ↓
CaseRepository
```

63. Database Session Management

Each request shall receive an appropriately scoped database session.

The application shall:

- commit explicitly
- rollback on failure
- close/release sessions reliably
- avoid long-running transactions

- avoid sharing sessions across concurrent requests

64. API Input Validation

Pydantic models shall validate all external input.

Examples:

UUID

Email

Date

DateTime

Enum

String length

File metadata

Pagination parameters

Validation errors shall return consistent API responses.

65. Business Validation

Business rules shall not be placed exclusively in Pydantic schemas.

For example:

Pydantic:

"due_at must be a valid datetime"

Service/domain:

"completed task cannot have a future completion timestamp"

This distinction shall be maintained.

66. Concurrency Control

The backend shall account for concurrent modification.

Potential mechanisms:

- optimistic locking
- version numbers
- row-level locking
- unique constraints

For important records, optimistic concurrency may use:

```
version = 7
```

and reject an update based on an obsolete version.

67. External Integrations

External services shall be accessed through infrastructure adapters.

Example:

```
ExternalIdentityProvider
```

```
ObjectStorageProvider
```

```
EmailProvider
```

```
SearchProvider
```

```
AIProvider
```

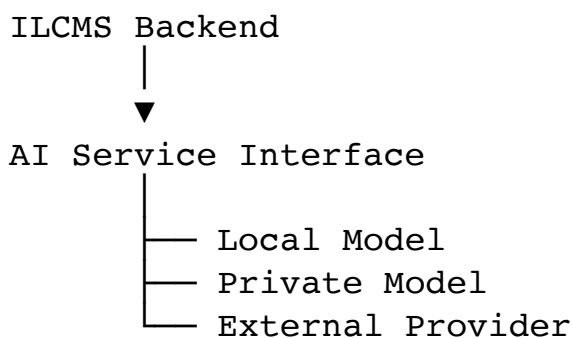
Business services shall not contain provider-specific HTTP implementation.

This allows providers to be replaced.

68. AI Integration Boundary

AI functionality shall be isolated from the core legal case domain.

Recommended architecture:



The AI subsystem shall receive only the minimum data necessary.

AI output shall not automatically become authoritative legal data without explicit user action.

AI-generated information shall be distinguishable from human-entered information.

69. AI Auditability

AI operations should record:

model identifier
model version
prompt/template version
requesting user
case/resource
timestamp
operation type
result status

Where legally and operationally appropriate, the system should retain sufficient metadata to reproduce or investigate an AI-assisted operation.

70. Document Text and AI

Extracted document text shall be treated as derived data.

The architecture shall distinguish:

Original document
Extracted text
OCR output
AI-generated summary
Human-approved summary
These must not be treated as equivalent sources of truth.

71. API Example — Create Case

Example:

POST /api/v1/cases
Content-Type: application/json
Authorization: Bearer <token>
Request:

```
{  
  "title": "Example legal matter",  
  "classification_id": "CIVIL",  
  "priority": "NORMAL"  
}
```

Response:

201 Created
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "case_number": "2026-00157",

```
  "title": "Example legal matter",
  "status": "OPEN"
}
```

The service shall also generate the corresponding audit event.

72. API Example — Document Upload

Initial request:

POST /api/v1/cases/{case_id}/documents

Response:

```
{
  "document_id": "...",
  "upload_url": "...",
  "expires_at": "...",
  "status": "UPLOAD_PENDING"
}
```

After upload:

UPLOAD_PENDING

↓

UPLOADED

↓

SCANNING

↓

PROCESSING

↓

READY

Failure:

PROCESSING

↓

FAILED

73. API Example — Search

GET /api/v1/search?q=Jónsson

Results shall contain only resources the authenticated user is authorized to see.

Authorization filtering shall occur server-side.

Search shall never be used as a mechanism for bypassing resource authorization.

74. API Rate Limiting

Rate limits shall be applied where appropriate, particularly to:

- authentication endpoints
- search
- exports
- expensive operations
- document-processing requests
- AI operations
- public integration endpoints

Rate limits shall be configurable per environment.

75. File Upload Limits

The backend shall enforce:

maximum file size
maximum request size
allowed document types
maximum number of files per operation
Limits shall be configurable.

Large-file processing shall be asynchronous.

76. Database Backup Awareness

The application shall not itself implement database backups.

Backups shall be managed by the infrastructure defined in v1.1.

However, application development shall ensure:

- migrations are reproducible
- data can be restored into a compatible schema
- seed/reference data is reproducible
- object-storage references remain consistent

77. Disaster Recovery

The backend shall support recovery procedures involving:

PostgreSQL restore

Object-storage recovery

RabbitMQ recovery

Redis reconstruction

Application redeployment

Migration execution

Search-index reconstruction

Redis and search indexes shall be considered rebuildable/derived where possible.

78. Reference Data

Reference data shall be separated from transactional data.

Examples:

Case statuses

Task statuses

Document types

Classification codes

Countries

Languages

Notification types

Reference data shall be version-controlled or managed through controlled migrations.

79. Icelandic Localization

Backend responses shall support localization where user-visible text is generated by the backend.

Localization keys should be used rather than hard-coded Icelandic/English messages.

Example:

`case.not_found`

`document.processing`

`task.completed`

The frontend should normally control presentation-language selection.

The backend must nevertheless preserve Icelandic legal names, document metadata and user-entered text exactly.

80. Character Encoding

UTF-8 shall be used throughout:

HTTP

JSON

PostgreSQL

Object metadata

Message queues

Search

Logs

Exports

The system shall be tested using Icelandic characters.

Example test values:

Þjóðskjalasafn Íslands

Hæstiréttur Íslands

Jón Þórðarson

Guðrún Ólafsdóttir

81. Data Privacy

Privacy shall be implemented as a system-wide engineering concern.

The backend shall support:

- least privilege
- access logging
- data minimization
- controlled exports
- retention configuration
- secure deletion where required
- encryption in transit
- encryption at rest
- access auditing

The precise legal retention period for a particular category of data shall be configurable rather than assumed by the application.

82. Production Deployment

The backend container shall:

- run as a non-root user
- contain only required runtime dependencies
- have a health check
- use a read-only filesystem where practical
- drop unnecessary Linux capabilities
- use resource requests/limits
- expose metrics where appropriate
- support graceful shutdown

83. Graceful Shutdown

The application shall correctly handle:

SIGTERM

Shutdown shall:

1. stop accepting new requests
2. finish appropriate in-flight work
3. close database connections
4. close messaging connections
5. close external clients
6. terminate cleanly

Workers shall similarly stop accepting new jobs before shutdown.

84. Security Headers

The application/API gateway shall support appropriate security headers.

Examples include:

Strict-Transport-Security

X-Content-Type-Options
Content-Security-Policy
Referrer-Policy

Exact implementation depends on ingress and frontend architecture.

85. Secrets

Application secrets shall be accessed through the deployment secret-management system.

The backend shall never print secrets during startup.

Configuration validation should report:

DATABASE_URL configured
OIDC configured
OBJECT_STORAGE configured
rather than printing their values.

86. Logging Requirements

Every significant log event should contain:

timestamp
level
service
environment
request_id
trace_id where available
event

Example:

```
{
  "level": "INFO",
  "event": "case_created",
  "request_id": "...",
  "case_id": "...",
  "user_id": "..."}
}
```

Sensitive content shall be excluded.

87. Error Handling

Expected business errors shall be represented by defined application exceptions.

Example:

CaseNotFoundError
AuthorizationError
InvalidCaseStateError
DocumentProcessingError
DuplicateCaseError

A centralized exception handler shall convert these into appropriate HTTP responses.

Unexpected errors shall:

- be logged
- receive a correlation ID
- return a generic client-facing error

88. API Compatibility

Changes shall follow compatibility rules.

Non-breaking:

Adding optional response fields
Adding new endpoints
Adding optional request fields
Potentially breaking:

Removing fields
Changing field types
Changing semantics
Changing required fields
Changing authentication behavior
Breaking changes require a new API version or migration strategy.

89. CI Quality Gates

The backend CI pipeline shall execute at minimum:

Dependency installation
↓
Formatting check
↓
Lint
↓
Type checking

↓

Unit tests

↓

Integration tests

↓

Security scanning

↓

Build

Coverage thresholds shall be introduced progressively rather than optimizing for an arbitrary percentage.

Critical business logic should have substantially higher coverage than infrastructure glue code.

90. Definition of Done — Backend

A backend feature is complete when:

- API endpoint implemented
- authorization implemented
- validation implemented
- business logic implemented
- database changes implemented
- migration created
- audit behavior implemented
- tests implemented
- OpenAPI documentation updated
- logging implemented
- metrics/tracing added where appropriate
- security implications reviewed
- Icelandic localization implications reviewed
- CI passes
- documentation updated

91. Initial Backend Development Sequence

Implementation should proceed in the following order.

Phase 1 — Foundation

Python project
FastAPI
configuration
logging
database
Alembic
health endpoints
CI

Phase 2 — Identity

OIDC integration
users
organizations
roles
permissions
authorization
tenant isolation

Phase 3 — Case Management

cases
parties
classification
case status
case assignment
audit

Phase 4 — Documents

documents
versions
object storage
signed uploads
virus scanning
processing pipeline
metadata
OCR
search indexing

Phase 5 — Workflow

tasks
deadlines

notifications
background jobs
RabbitMQ

Phase 6 — Search

case search
party search
document metadata
full-text search
Icelandic language handling

Phase 7 — Advanced Features

exports
reporting
integrations
AI subsystem
advanced workflow

92. Initial MVP Backend

The first production-capable backend MVP should include:

Identity

- authentication
- users
- organizations
- RBAC
- tenant isolation

Cases

- create
- read
- update
- assign
- close
- classify

Parties

- create
- update
- associate with cases
- search

Documents

- upload
- metadata
- versioning
- download
- processing status
- authorization

Tasks

- create
- assign
- complete
- deadlines

Audit

- immutable event creation
- audit retrieval for authorized users

Search

- cases
- parties
- documents
- metadata

Infrastructure

- PostgreSQL

- object storage
- RabbitMQ
- Redis
- Kubernetes deployment

93. Backend Security Baseline

Before production, the backend must pass the following minimum checks:

```
[ ] No hard-coded credentials
[ ] Dependency vulnerability scan clean or risk accepted
[ ] Container scan completed
[ ] SAST completed
[ ] Authentication tested
[ ] Authorization tested
[ ] Tenant isolation tested
[ ] IDOR/BOLA tests completed
[ ] File-upload security tested
[ ] Audit logging verified
[ ] Sensitive logging reviewed
[ ] TLS verified
[ ] Database encryption verified
[ ] Object-storage permissions verified
[ ] Backup/restore tested
[ ] Disaster recovery procedure tested
```

94. Important Legal/Records-Management Boundary

ILCMS shall provide technical capabilities that can support Icelandic legal and records-management requirements.

However, the software shall not encode assumptions that every organization has identical legal obligations.

In particular, configurable mechanisms should be used for:

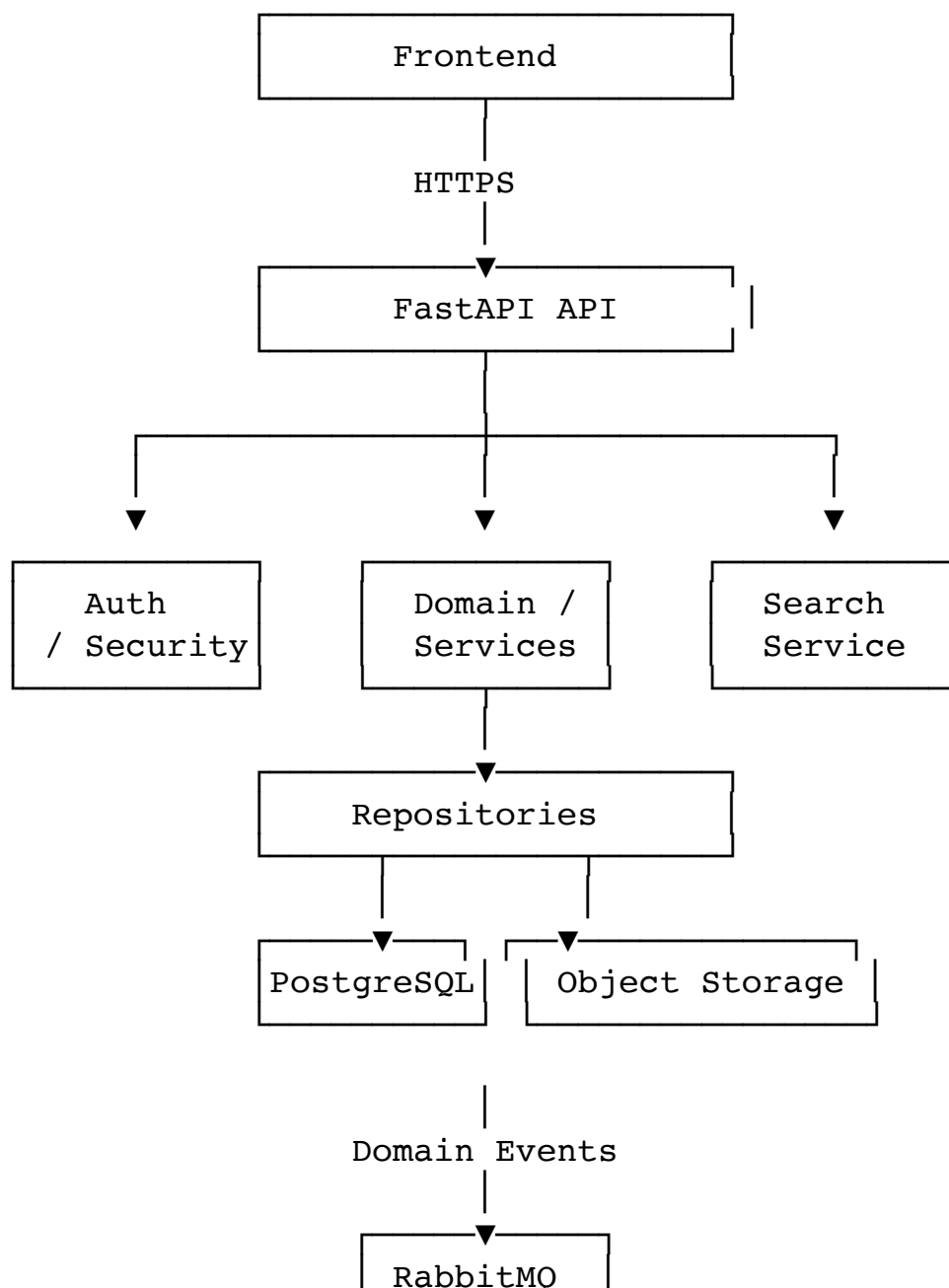
- classification systems
- retention periods
- document types

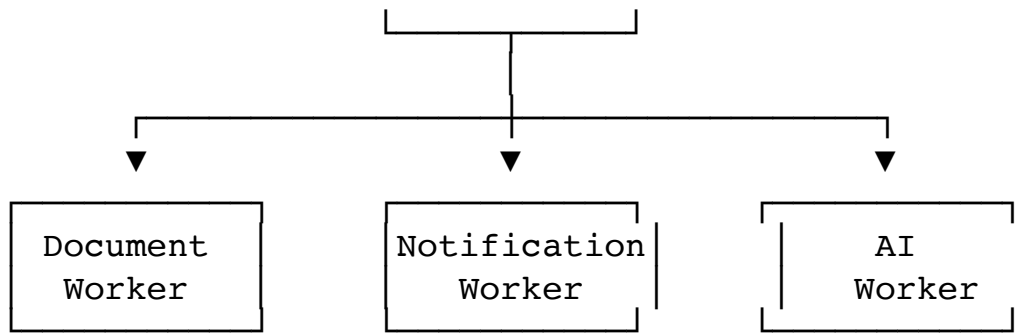
- case statuses
- access policies
- archival/export workflows
- destruction procedures

The implementation shall be reviewed against the current requirements of the relevant Icelandic authorities and legislation before production deployment.

95. Architectural Objective

The resulting backend should provide this logical architecture:





96. Version 1.2 Acceptance Criteria

Technical Specification v1.2 is considered implemented when:

1. The backend project structure is established.
2. FastAPI is operational.
3. PostgreSQL integration is operational.
4. Alembic migrations are operational.
5. Authentication and authorization foundations are operational.
6. Tenant isolation is enforced.
7. Case and party domains are implemented.
8. Document metadata and object storage integration are implemented.
9. Background processing infrastructure is operational.
10. Audit events are generated for defined sensitive operations.
11. API versioning is established.
12. OpenAPI documentation is generated.
13. Unit and integration test frameworks are operational.
14. CI quality gates are enforced.
15. Structured logging and observability are operational.
16. Security testing covers cross-tenant access and object-level authorization.
17. Icelandic UTF-8 data and naming conventions are tested.
18. The backend can be deployed using the Kubernetes architecture defined in v1.1.

97. Next Specification

The recommended next step is:

Technical Specification v1.3 — Database & Data Model Specification

v1.3 should translate the backend domain model into a concrete PostgreSQL design, including:

- complete entity model
- table definitions
- columns and data types
- primary/foreign keys
- indexes
- constraints
- tenant isolation
- audit schema
- document/version schema
- case/party relationships
- task/deadline model
- classification/málalykill model
- workflow/state transitions
- retention metadata
- database-level security
- Alembic migration strategy
- sample SQL
- ER diagram
- seed/reference data
- data-integrity rules

This should be the next implementation document because the database schema becomes the foundation for the FastAPI repositories, services, migrations, tests, and ultimately the frontend API contracts.

That gives us a concrete backend implementation contract rather than just an architectural overview. **v1.3 should logically go quite deep into the PostgreSQL schema and ER model**, because that will let us start turning the specification into actual code.

